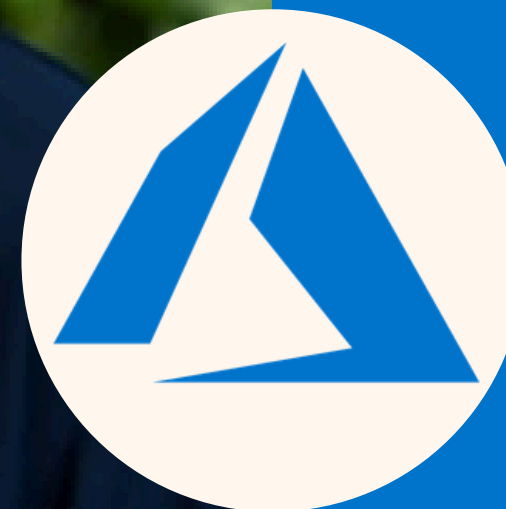


 **GET STARTED TODAY**

LUKE

GINN



**AZURE
CLOUD
MASTERY**



Unit 13 covered HITL approvals. Unit 14 covers Foundry Trace – OpenTelemetry-based debugging for agent reasoning loops and failures.

- **Review of Unit 13:** You learned to pause agents for human approval. Now you need to debug when agents get stuck in reasoning loops or call wrong tools.
- **What This Unit Covers:** You will learn OpenTelemetry (a tracing standard), Foundry Trace (Microsoft's implementation), spans (individual operations), and attributes (metadata).
- **Why Tracing Matters:** Beta exam candidates fail on reasoning loop questions. Tracing is the difference between pass and fail.

Exam Takeaway: Foundry Trace is 20% of the exam (Observability domain). Reasoning loops are a common failure point – tracing solves them.



What Is Distributed Tracing?

Distributed tracing records every operation an agent performs – LLM calls, tool calls, memory lookups – as a timeline of spans with timing and metadata.

- **Trace Definition:** A trace is a complete record of a single agent conversation, from first user message to final response. It contains many spans (individual operations).
- **Span Definition:** A span represents one operation: an LLM call (duration 500ms), a tool call (duration 200ms), or a memory lookup (duration 5ms).
- **Why Distributed:** Agent operations span multiple services (LLM, database, search). Tracing connects them into a single timeline.

Exam Takeaway: Traces = full conversation. Spans = individual operations. The exam tests reading spans to find where agents fail.



What Is OpenTelemetry?

OpenTelemetry is an open standard that defines how applications create, format, and send traces to monitoring systems like Foundry Trace.

- **OpenTelemetry Definition:** OpenTelemetry (OTel) is a vendor-neutral API, SDK, and protocol for collecting traces. Microsoft Foundry implements this standard.
- **Why OpenTelemetry Matters:** Because OTel is a standard, you can use the same tracing code whether you send traces to Foundry, Azure Monitor, or third-party tools.
- **OTel Components:** The OTel SDK provides ``Tracer`` (creates spans), ``Span`` (records operation), and ``SpanProcessor`` (exports spans to Foundry Trace).

Exam Takeaway: OpenTelemetry is the industry standard. Foundry Trace implements OTel. The exam tests OTel concepts, not Foundry-specific proprietary formats.



Foundry Trace – Microsoft's Tracing Implementation

AZURE
CLOUD
MASTERY

Foundry Trace is Microsoft's fully managed distributed tracing service, integrated with Azure Monitor, that visualizes agent execution as interactive timelines.

- **Foundry Trace Definition:** Foundry Trace automatically captures traces from agents running in Foundry Agent Service. You can also send custom spans from code.
- **Integration with Azure Monitor:** Foundry Trace sends trace data to Azure Monitor Log Analytics. You query traces using KQL (Kusto Query Language).
- **Trace Visualization:** In Foundry portal, you select a conversation ID and see an interactive waterfall diagram of every span, its duration, and its attributes.

Exam Takeaway: Foundry Trace is the UI and storage. OpenTelemetry is the protocol. The exam tests interpreting Foundry Trace waterfall diagrams.



Reasoning Loop – The Most Common Agent Failure

A reasoning loop occurs when an agent repeats the same action over and over, making no progress, because it lacks information or misinterprets results.

- **Loop Example:** Agent calls Search Tool with query "refund policy". Receives results. Calls again with same query. Receives same results. Never proceeds to answer.
- **Why Loops Happen:** The LLM does not see a required field in the results, so it calls the tool again with identical parameters, expecting different results.
- **Cost of Loops:** Each loop iteration costs tokens, time, and API calls. A loop of 10 iterations costs 10x the normal execution.

Exam Takeaway: Reasoning loops are the top cause of exam failure. Foundry Trace shows you exactly where the loop starts and why.



Detecting Loops in Foundry Trace

In Foundry Trace, a reasoning loop appears as a repeating sequence of identical spans – same operation name, same parameters – with no progress span between them.

- **Loop Signature:** Span A (Search Tool call) → Span B (Search Tool call) → Span C (Search Tool call). All three have same `query` attribute. No answer span.
- **Duration Pattern:** Each loop iteration takes the same amount of time. If three spans each take 500ms, total 1500ms wasted before failure.
- **Identifying the Cause:** Examine span attributes. If search results contain no `policy\_text` field but agent expects it, the agent loops because it never sees the field.

Exam Takeaway: Repeated identical spans without progress indicate a reasoning loop. The exam tests identifying loops by pattern matching in trace waterfalls.



Span Attributes – Recording Metadata

Span attributes are key-value pairs you add to a span to record context: tool name, query string, HTTP status code, error message, or any relevant metadata.

- **Attribute Definition:** Attributes are structured metadata. Example:
``span.set_attribute("tool.name", "search_sharepoint")`, `span.set_attribute("query", "budget report")`.`
- **Required Attributes for Exam:** ``agent.name``, ``operation.type`` (llm_call, tool_call, handoff), ``duration.ms``, ``status`` (success/error), ``error.message`` (if error).
- **Searching by Attributes:** In Foundry Trace, filter traces by ``attributes.tool.name == "search_sharepoint"`` to find all calls to a specific tool.

Exam Takeaway: The coding expectation is adding attributes to spans. The exam tests which attributes to add for different operations (tool calls, LLM calls, handoffs).



Your agent code creates a custom span using the OpenTelemetry SDK, records attributes, and ends the span when the operation completes.

- **Create Tracer:** ``from opentelemetry import trace; tracer = trace.get_tracer("my.agent")``.
- **Start Span:** ``with tracer.start_as_current_span("tool_call") as span:`
`span.set_attribute("tool.name", "search")`` then execute tool.
- **Record Result:** After tool completes, add ``span.set_attribute("result_length", len(result))``. If error, ``span.set_attribute("error", True)``.
- **Span Ends Automatically:** When ``with`` block exits, span ends and sends to Foundry Trace.

Exam Takeaway: Use ``with tracer.start_as_current_span()`` to create spans. Set attributes before the operation completes. The exam tests this pattern.



Instrumenting LLM Calls with Spans

Each LLM call should have its own span recording prompt tokens, response tokens, model name, and latency.

- **Span Name:** `"llm_call"`. Attributes: ``model_name`` (gpt-4o), ``prompt_tokens`` (from response usage), ``completion_tokens``, ``total_tokens``, ``latency_ms``.
- **Recording Tokens:** After LLM response, extract ``response.usage.prompt_tokens``, add as span attributes.
- **Error Recording:** If LLM returns error (429 rate limit, 500 server error), add ``error=true``, ``error.type=http_error``, ``error.message="Rate limit exceeded"``.

Exam Takeaway: LLM spans must record token usage and error details. The exam tests extracting token counts from LLM response JSON.

Instrumenting Tool Calls with Spans

Each tool call requires a span recording the tool name, parameters, HTTP status code, response size, and any error.

- **Span Name:** `"tool_call"`. Attributes: `tool.name`, `tool.parameters` (JSON string), `http.status_code`, `response.size_bytes`.
- **Before Call:** Start span, add `tool.name` and `tool.parameters`. Execute REST call.
- **After Call:** Add `http.status_code`. If 200, add `response.size_bytes`. If 4xx/5xx, add `error=true` and `error.message`.
- **Duration:** OpenTelemetry automatically records span duration. No manual timing needed.

Exam Takeaway: Tool call spans must capture parameters, status code, and response size. The exam tests attribute selection for debugging tool failures.

Instrumenting Handoffs with Spans

In multi-agent systems, each handoff between agents should have a span recording `from_agent`, `to_agent`, and state size.

- **Span Name:** `"handoff"`. Attributes: ``from_agent`` (name of handing-off agent), ``to_agent`` (receiving agent), ``conversation_state_size_bytes``.
- **Before Handoff:** Start span, add ``from_agent`` and ``to_agent``. Calculate state size with ``len(json.dumps(state))``.
- **After Handoff:** Add ``success=true`` if handoff completed, or ``error=true`` if receiving agent rejected or timed out.
- **Trace Visualization:** Foundry Trace shows handoffs as arrows connecting agent spans. Loops appear as repeated handoffs between same agents.

Exam Takeaway: Handoff spans require `from_agent`, `to_agent`, and state size. The exam tests reading handoff traces to find loop patterns.



Span Events – Recording Discrete Moments

Span events are timestamped records within a span for discrete moments like sending a request, receiving headers, or parsing JSON.

- **Event Definition:** Events are point-in-time records. Unlike attributes (set once), events can occur multiple times within a span with different timestamps.
- **Adding an Event:** ``span.add_event("request_sent", attributes={"url": endpoint, "method": "POST"})``.
- **When to Use Events:** Use events for logging within an operation. Example: "sending request", "received response", "starting parsing", "parsing complete".
- **Events vs. Attributes:** Attributes describe the span outcome. Events describe the span progress.

Exam Takeaway: Events are for progress logging. Attributes are for final outcome. The exam tests choosing events vs. attributes for different debugging needs.



Querying Traces with KQL

You query Foundry Trace data in Azure Monitor Log Analytics using Kusto Query Language (KQL) to find failed tool calls or slow LLM responses.

- **KQL Definition:** KQL is the query language for Azure Monitor. You write queries to filter, aggregate, and visualize trace data.
- **Find Failed Tool Calls:** ``traces | where span.name == "tool_call" | where attributes.error == true | project timestamp, attributes.tool.name, attributes.error.message``.
- **Find Slow LLM Calls:** ``traces | where span.name == "llm_call" | where duration > 2000 | project timestamp, attributes.model_name, duration, attributes.total_tokens``.
- **Time Range:** Always include a time filter: ``| where timestamp > ago(1h)`` to limit to last hour.

Exam Takeaway: KQL queries filter traces by span name, attributes, and duration. The exam tests writing basic KQL to find errors or slow operations.



Trace Sampling – Reducing Overhead

Trace sampling means recording only a percentage of traces (e.g., 10%) to reduce performance overhead and storage costs in high-volume agents.

- **Why Sample:** Recording every trace for an agent handling 1 million conversations per day creates massive overhead and storage costs.
- **Head-Based Sampling:** Decision made at trace start. Example: ``TraceIdRatioBased(0.1)`` records 10% of traces. The same trace either fully recorded or fully skipped.
- **Tail-Based Sampling:** Record all traces briefly, then decide which to keep based on criteria (e.g., keep only traces with errors). More accurate but more complex.
- **Configuring in Foundry:** In Foundry Project settings, set sampling rate. Default 100% for development, 10% for production.

Exam Takeaway: Sampling reduces overhead. Head-based (fixed percentage) is simpler. Tail-based (keep errors) is smarter. The exam tests sampling tradeoffs.



Debugging Workflow with Foundry Trace

The debugging workflow starts with a user report of agent failure, then searching traces by conversation ID or error flag, then examining spans to find root cause.

- **Step 1 – Identify Failure:** User reports "Agent gave wrong answer at 3:15 PM on Tuesday for conversation XYZ." Find conversation ID in your logs.
- **Step 2 – Search Trace:** In Foundry Trace, search by ``conversation_id == "XYZ"``. Load the complete trace waterfall.
- **Step 3 – Locate Error Span:** Find the first span with ``error=true`` or unusually long duration. Examine its attributes and events.
- **Step 4 – Fix and Verify:** Fix code, redeploy, reproduce user request, verify trace shows no error.

Exam Takeaway: The exam tests the debugging workflow: find conversation ID, load trace, locate error span, examine attributes, fix, verify.



Unit 14 Summary – Foundry Trace Debugging

AZURE CLOUD MASTERY

You have learned to instrument agents with OpenTelemetry spans, detect reasoning loops in trace waterfalls, and query traces with KQL.

- **Core Concepts:** Traces (full conversation), spans (individual operations), attributes (metadata), events (point-in-time logs). OpenTelemetry is the standard.
- **Critical for Exam:** Reasoning loops appear as repeated identical spans. Foundry Trace visualizes loops as repeating blocks without progress.
- **Coding Skills:** Create spans with ``tracer.start_as_current_span()``. Add attributes for tool name, parameters, status, error, and token usage.
- **Query Skills:** KQL queries filter traces by ``span.name``, ``attributes.error``, and ``duration > threshold``.
- **Next Unit Preview:** Unit 15 introduces Performance Evaluation – measuring tool call accuracy, task adherence, and intent resolution.

Exam Takeaway: Foundry Trace is 20% of the exam. Master loop detection in waterfall diagrams, span attributes, and KQL queries for debugging. This is the difference between pass and fail.

**THANKS
FOR
WATCHING**

